BSI Test Dashboard

User Manual

Black Sphere Industries

Version 1.0 — March 2026

Table of Contents

1 Introduction . . . 3
  1.1 Key Capabilities . . . 3
2 Getting Started . . . 3
  2.1 Prerequisites . . . 3
  2.2 Accessing the Dashboard . . . 3
  2.3 Authentication . . . 3
  2.4 First Steps . . . 3
3 Features . . . 3
  3.1 Overview Dashboard . . . 3
  3.2 Test Runs . . . 4
  3.3 Run Detail . . . 4
  3.4 Manifest Explorer . . . 4
  3.5 Coverage Map . . . 4
  3.6 Trends & Analytics . . . 4
  3.7 Settings . . . 5
4 User Interface . . . 5
  4.1 Layout . . . 5
  4.2 Navigation . . . 5
  4.3 Theme Support . . . 5
  4.4 Badges and Color Coding . . . 5
5 Workflows . . . 5
  5.1 Importing a Test Run . . . 5
  5.2 Investigating a Test Failure . . . 6
  5.3 Comparing Two Runs . . . 6
  5.4 Checking Release Readiness . . . 6
6 Architecture . . . 6
  6.1 Sequence Diagram . . . 7
  6.2 State Diagram . . . 9
  6.3 Technology Stack . . . 9
7 Configuration . . . 10
  7.1 Firebase Project . . . 10
  7.2 Theme Configuration . . . 10
8 Troubleshooting . . . 10
  8.1 Cannot Sign In . . . 10
  8.2 No Data Displayed . . . 10
  8.3 Charts Not Rendering . . . 10
  8.4 Import Validation Fails . . . 10
  8.5 Incorrect Pass Rate . . . 10

1 Introduction

The BSI Test Dashboard is a web-based integration test monitoring and analytics platform built for Black Sphere Industries. It aggregates test run results across the entire BSI application portfolio, providing real-time visibility into release health, test coverage, and quality trends.

The dashboard consumes test result JSON payloads produced by the BSI integration test runner and stores them in Firebase Firestore. It is designed for engineering teams who need to track pass rates, identify flaky tests, and verify release readiness across 30+ browser-based applications.

1.1 Key Capabilities

- Centralized test result aggregation across the BSI app portfolio
- Real-time pass rate tracking with historical trend analysis
- Test coverage mapping across all BSI repositories
- Flaky test detection through cross-run analysis
- Release manifest inspection with pinned commit hashes
- Suite-level and test-level drill-down reporting

2 Getting Started

2.1 Prerequisites

- A Google account authorized in the BSI Firebase project (`arnout-s-homelab`)
- A modern web browser (Chrome, Firefox, Safari, Edge)

2.2 Accessing the Dashboard

The BSI Test Dashboard is hosted on Cloudflare Pages and accessible through the BSI portal at `blacksphereindustries.nl`. It can also be opened directly via its standalone URL.

2.3 Authentication

The dashboard requires Google sign-in via Firebase Authentication. On launch, a login screen is presented with a "Sign in with Google" button. After successful authentication, the user's avatar and email are displayed in the sidebar, and all dashboard sections become available.

2.4 First Steps

1. Sign in with your Google account
2. Review the **Overview** section for the latest pass rate and run count
3. Navigate to **Runs** to browse individual test run results
4. Check the **Coverage Map** to identify untested repositories
5. Use **Settings** to import new test run JSON data

3 Features

3.1 Overview Dashboard

The Overview section provides a high-level summary of test health:

- **Latest Pass Rate** — the percentage of passing tests in the most recent run, displayed as a large KPI card
- **Total Test Runs** — cumulative count of all imported test runs
- **Manifest Version** — the current release manifest version string

Two charts are displayed:

- **Pass Rate Trend** — a line chart (Chart.js) showing pass rate evolution over time
- **Suite Health** — a bar chart summarizing pass/fail/skip counts by test suite

Quick action buttons provide one-click access to the integration test runner and the full runs list. A **Recent Events** feed shows the latest test-related activity.

3.2 Test Runs

The Runs section displays a sortable, searchable table of all test runs with the following columns:

- **Date** — timestamp of the test execution
- **Version** — the manifest/release version under test
- **Mode** — `demo` or `live`, indicating whether the run targeted demo mode or production
- **Pass / Fail / Skip** — count of tests in each result category
- **Rate** — overall pass rate, color-coded green (>=90%), amber (>=70%), or red (<70%)
- **Duration** — total elapsed time for the run

The table supports:
- **Text search** by version or date
- **Mode filters** — All, Demo, Live
- **Column sorting** — click any header to sort ascending/descending
- **Run comparison** — select two runs via checkboxes and click "Compare Selected"

3.3 Run Detail

Clicking a row in the Runs table navigates to a detailed view showing:

- A pass/fail summary bar with total counts
- Collapsible test suite sections, each containing individual test results
- Per-test status icons (pass, fail, skip), names, timing data, and error messages for failures

3.4 Manifest Explorer

The Manifest section displays the release manifest embedded in test runs. This includes:

- Pinned repository names and commit hashes for each BSI app
- Test coverage status per repository
- The manifest version string

This allows engineers to verify exactly which code was tested in a given run.

3.5 Coverage Map

The Coverage Map provides a matrix view of test suite coverage across all BSI repositories:

- **Apps Tested** — count of repositories with at least one test suite
- **Not Tested** — count of repositories without test coverage
- **Coverage Percentage** — overall portfolio coverage

A visual coverage matrix shows tested vs. untested cells for each app/suite combination. An "Untested Repos" section lists all repositories that lack integration tests.

3.6 Trends & Analytics

The Trends section provides deeper analytical views:

- **Pass Rate Over Time** — line chart with more granular time-series data
- **Duration Trend** — tracks test execution time over successive runs

- **Failure Heatmap** — a color-coded grid showing suites (rows) vs. recent runs (columns), with cells colored green (pass), red (fail), or gray (skip)
- **Suite Stability** — a bar chart ranking suites by consistency across runs
- **Flaky Test Detection** — identifies tests that alternate between pass and fail across multiple runs, flagging them for investigation

3.7 Settings

The Settings section provides data management capabilities:

- **Import Test Run** — paste test run JSON exported from the integration runner. A two-step process validates the JSON structure before submitting it to the dashboard.
- **Data Management** — options for managing stored test data

4 User Interface

4.1 Layout

The dashboard uses a sidebar + main content layout:

- A **fixed sidebar** (240px wide) on the left contains navigation links and a theme toggle
- The **main content area** displays the selected section
- On mobile (< 768px), the sidebar collapses to a horizontal bar above the content

4.2 Navigation

Sidebar navigation items:

- Overview
- Runs
- Run Detail (appears only after selecting a run)
- Manifest
- Coverage Map
- Trends
- Settings

4.3 Theme Support

A "Toggle Theme" button in the sidebar switches between dark mode (amber/zinc palette) and light mode (deep blue #10069f palette). Theme preference persists across sessions.

4.4 Badges and Color Coding

Status badges use a consistent color scheme:

- **Amber** — primary accent, highlights, and branding
- **Green** — pass status, healthy metrics
- **Red** — fail status, critical alerts
- **Blue** — informational badges
- **Purple** — special categories
- **Zinc/Gray** — muted, skipped, or inactive states

5 Workflows

5.1 Importing a Test Run

1. Navigate to **Settings** > **Import Test Run**
2. In the BSI integration test runner, click "Export for Dashboard" to copy the JSON payload

3. Paste the JSON into the text area
4. Click **Validate** to check the JSON structure
5. If validation passes, click **Submit to Dashboard**
6. Navigate to **Overview** or **Runs** to see the new data

5.2 Investigating a Test Failure

1. Go to **Overview** and check the latest pass rate
2. If the rate is below target, click **View All Runs** or navigate to **Runs**
3. Click the failing run row to open **Run Detail**
4. Expand the failing test suite to see individual test results
5. Review the error message and timing data for each failed test
6. Use the **Trends** > **Flaky Test Detection** to check if the failure is intermittent

5.3 Comparing Two Runs

1. Navigate to **Runs**
2. Select two runs using the checkboxes in the leftmost column
3. Click the **Compare Selected** button that appears
4. Review the side-by-side comparison of pass rates, suite results, and duration

5.4 Checking Release Readiness

1. Navigate to **Manifest** to verify the expected repository commits are pinned
2. Navigate to **Coverage Map** to ensure all critical apps have test coverage
3. Navigate to **Overview** and confirm the latest pass rate meets the release threshold
4. Review **Trends** for any negative pass rate trends or increasing flakiness

6 Architecture

The BSI Test Dashboard is a single-file HTML application with inline JavaScript and CSS. It uses Firebase for authentication and data persistence, and Chart.js for data visualization.

6.1 Sequence Diagram

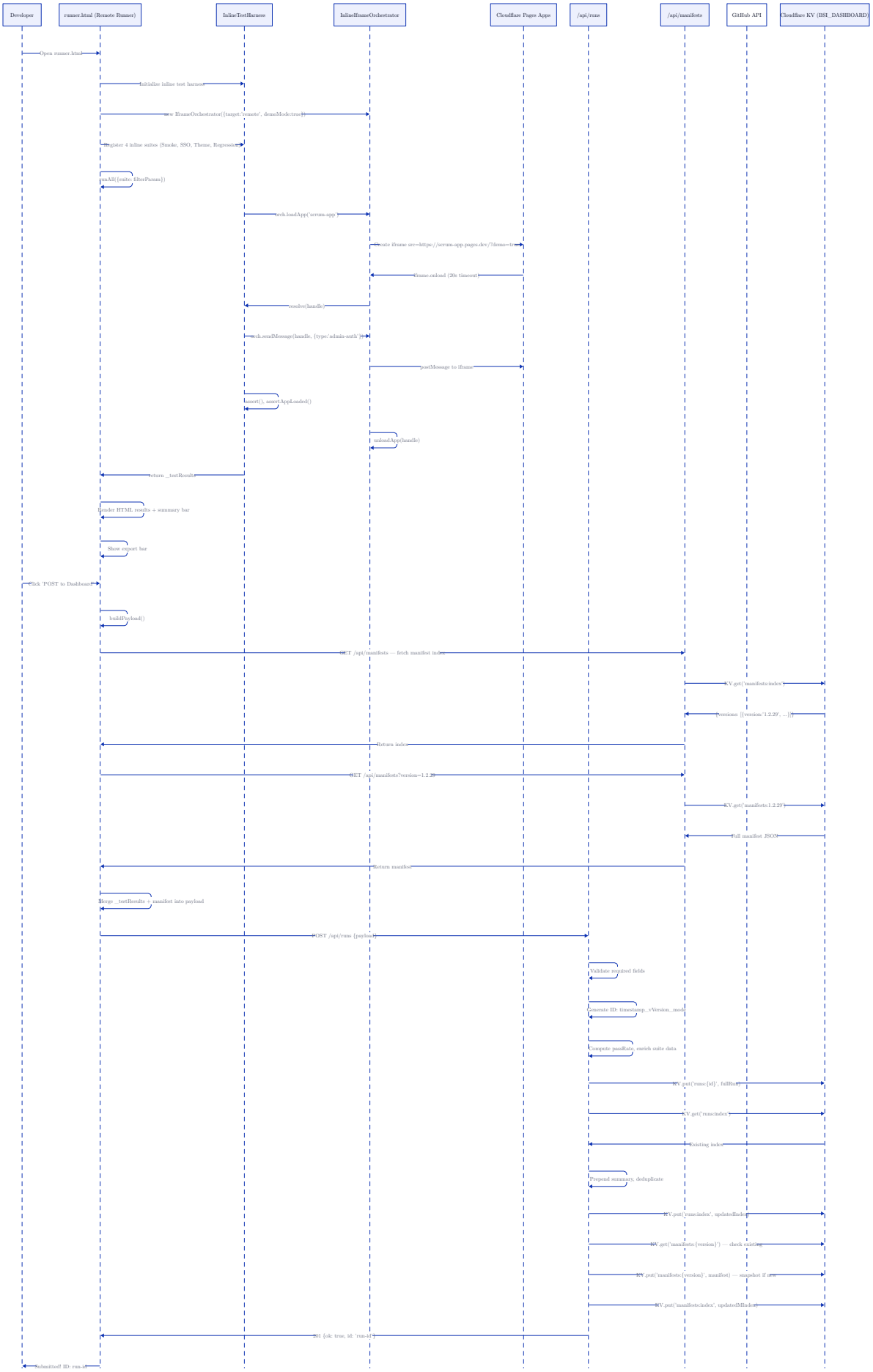

Figure 1: Main interaction sequence for importing and viewing test runs

6.2 State Diagram

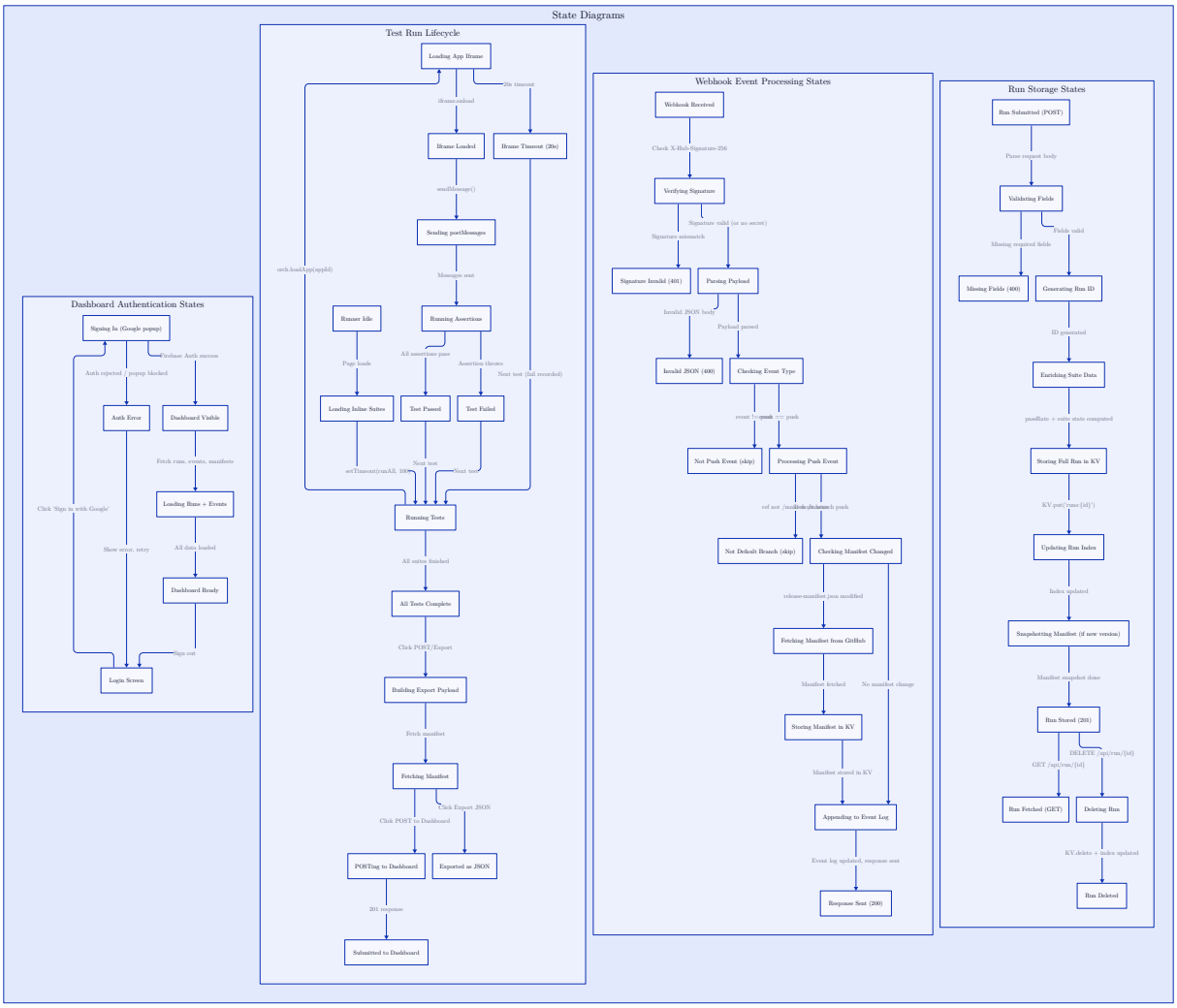

Figure 2: Application state transitions

6.3 Technology Stack

Component	Technology
UI Framework	Vanilla JavaScript with Tailwind CSS
Fonts	Inter (UI), JetBrains Mono (code/data)
Icons	Lucide Icons
Charts	Chart.js
Authentication	Firebase Auth (Google sign-in)
Database	Firebase Firestore
Hosting	Cloudflare Pages

7 Configuration

7.1 Firebase Project

The dashboard connects to the `arnout-s-homelab` Firebase project. Firebase configuration is embedded in the application source code. The dashboard uses:

- **Firebase Auth** — Google sign-in provider for user authentication
- **Firestore** — document database for storing test run data, manifest information, and user preferences

7.2 Theme Configuration

The dashboard supports two themes configured via CSS custom properties:

- **Dark mode** (default): zinc backgrounds with amber accent (#f59e0b)
- **Light mode**: white/light backgrounds with deep blue accent (#10069f)

Theme selection is toggled via the sidebar button and persisted in the browser.

8 Troubleshooting

8.1 Cannot Sign In

- Verify your Google account is authorized in the Firebase project
- Check that third-party cookies are enabled in your browser
- Try clearing browser cache and signing in again
- If using the app in an iframe (portal mode), ensure the portal is authenticated first

8.2 No Data Displayed

- Navigate to **Settings** and import at least one test run JSON
- Verify the JSON was exported from a compatible version of the integration runner
- Check the browser console for Firestore connection errors

8.3 Charts Not Rendering

- Ensure Chart.js CDN (`cdn.jsdelivr.net`) is accessible
- Check for browser extensions that may block CDN resources
- Try refreshing the page or switching to a different browser

8.4 Import Validation Fails

- Ensure the JSON contains the required fields: `timestamp`, `version`, `suites` array
- Each suite must have a `name` and `tests` array
- Each test must have `name` and `status` (pass/fail/skip) fields
- Check for trailing commas or other JSON syntax errors

8.5 Incorrect Pass Rate

- Verify that skipped tests are not being counted as failures
- Check if demo-mode and live-mode runs are being conflated — use the mode filter
- Re-import the test run if data corruption is suspected